

ELIS_Vulcan-90

Электронная Логическая Интеллектуальная Система «Вулкан-90»

SCADA для реального мира

РУКОВОДСТВО АДМИНИСТРАТОРА

Версия документа: 1.0
Дата: 17 апреля 2026 г.

СОДЕРЖАНИЕ

1.1. О системе ELIS_Vulcan-90 ……..5
1.2. Назначение документа ……..5
1.3. Целевая аудитория ……5
1.3.1. Промышленные и системные администраторы ……..6
1.3.2. Инженеры АСУ ТП и КИПиА ……..6
1.3.3. DevOps-инженеры и SRE ……..6
1.3.4. Студенты профильных вузов ……...7
1.3.5. Научные лаборатории и исследовательские центры ……7
1.3.6. Малый и средний бизнес ……..8
1.3.7. Open-source сообщество ……...9
1.3.8. Универсальность документа ……9
1.3.9. Миссия и ценности проекта ……10

Раздел 1. Архитектура и состав системы ……11
1.1. Структура директорий проекта ……..11
1.2. Функциональные модули ……13
1.3. Карта портов ……14
1.4. Критичность данных ……...15

Раздел 2. Требования и подготовка сервера ……15
2.1. Аппаратные требования ……..15
2.2. Программные требования ……...17
2.3. Подготовка Ubuntu Server ……...18
2.4. Настройка сети и статического IP ……..20

Раздел 3. Первичная установка ……22
3.1. Установка Docker и Docker Compose ……23
3.2. Развертывание проекта из архива ……..24
3.3. Создание файла .env ……25
3.4. Настройка udev для PLC ……27
3.5. Первичный запуск стека ……29

Раздел 4. Автоматическая установка через скрипты ……31
4.1. Полная установка одной командой ……31
4.2. Пошаговая установка (для отладки) ……..32
4.3. Флаги установщика ……33
4.4. Что делает каждый скрипт ……33
4.5. Проверка после установки ……34

Раздел 5. Управление жизненным циклом контейнеров ……...36
5.1. Основные команды ……...36
5.2. Изоляция контейнеров ……36
5.3. Работа с томами ……36

Раздел 6. Резервное копирование ……...37
6.1. Критически важные данные ……37
6.2. Ручное создание полного бэкапа ……37
6.3. Создание образов всех контейнеров (docker save) ……38
6.4. Автоматический бэкап через cron ……...40
6.5. Проверка целостности бэкапа ……41
6.6. Ротация бэкапов ……42
6.7. Бэкап «золотого слепка» образов ……42

Раздел 7. Восстановление системы ……..43
7.1. Восстановление из полного бэкапа ……..43
7.2. Скрипт restore.sh — автоматизация прав ……...44
7.3. Восстановление после потери сервера ……45
7.4. Перенос на другой сервер через флешку ……47
7.5. Перенос через образы контейнеров ……..47

Раздел 8. Удаление системы ……..48
8.1. Три режима удаления ……48
8.2. Мягкое удаление (--keep-data) ……..48
8.3. Полное удаление ……48
8.4. Проверка после удаления ……...49
8.5. Если uninstall.sh упал на Permission denied ..…….. 49

Раздел 9. Мониторинг и диагностика ……...50
9.1. Uptime Kuma ……50
9.2. Apprise — центр уведомлений ……51
9.3. Диагностические команды ……..51

Раздел 10. Управление пользователями и безопасность ……..52
10.1. Пользователи FUXA ……52
10.2. Пользователи Portainer ……52
10.3. Базовая авторизация Draw.io ……...53
10.4. Общие рекомендации ……...53
10.5. Чек-лист безопасности ……54

10.6. Чек-лист при передаче сервера ……...54

Раздел 11. Обновление системы ……54
11.1. Обновление образов ……..54

11.2. Обновление Python-модуля …….55
11.3. Обновление «золотого слепка» образов …….55

Раздел 12. Типовые неисправности ……..55
12.1. FUXA не видит PLC …….55
12.2. FUXA не подключается к MQTT ……56
12.3. FUXA Cannot GET /userRoles ……..56
12.4. Telegraf не пишет в InfluxDB ……..56
12.5. Grafana Permission denied ……56
12.6. Portainer не даёт создать админа ……57
12.7. Контейнер не запускается из-за devices ……57
12.8. Стек падает из-за занятого порта ……58
12.9. install.sh падает на generate_password (Exit 141) ……...58
12.10. Проблемы с правами после tar ……..58
12.11. Не работает USB-монтирование ……...58

Раздел 13. Приложения ……..60
13.1. Быстрые команды ……60
13.2. URL всех сервисов ……...60
13.3. Чек-лист полного цикла ……...60
13.4. Глоссарий ……..60
13.5. Контакты и поддержка ……60

Введение

1.1. О системе ELIS_Vulcan-90

Электронная Логическая Интеллектуальная Система «Вулкан-90» (ELIS_Vulcan-90) — современная высокотехнологичная SCADA-система нового поколения, разработанная для обеспечения сквозного контроля, аналитики и беспрецедентной надежности технологических процессов в реальном мире. Реализуя концепцию «от датчика до решений», система объединяет классические инструменты HMI, технологии промышленного интернета вещей (IIoT), продвинутую аналитику данных и алгоритмы автоматизации.

Архитектура ELIS_Vulcan-90 построена на принципах микросервисной контейнеризации, гибкости, простоты развертывания и непрерывного развития. Экосистема состоит из **12 изолированных функциональных блоков**, развернутых в виде независимых Docker-контейнеров под управлением единого конфигурационного файла docker-compose.yml.

1.2. Назначение документа

Настоящее руководство — **полная практическая инструкция** по развертыванию, эксплуатации, резервному копированию, обновлению и удалению всей экосистемы ELIS_Vulcan-90. Оно написано на основе **реального опыта установки и тестирования** и содержит **точные команды**, проверенные на Ubuntu Server 22.04 LTS.

Настоящий документ сфокусирован на инфраструктурной части и оркестрации контейнеров. Подробное конфигурирование, проектирование интерфейсов и внутренняя настройка отдельных сервисов не входят в рамки данного руководства.

Для функциональных модулей системы предусмотрены отдельные специализированные руководства:

Слой сбора данных, визуализации и аналитики (SCADA/HMI/TSDB):

- **HMI-Frontend (FUXA)** — Руководство оператора по созданию мнемосхем, привязке тегов АСУ ТП и управлению DAQ-интенсивами.
- **HMI-Designer (Draw.io)** — Инструкция по проектированию технологической графики и импорту кастомных библиотек символов.

- **InfluxDB** — Руководство по управлению базами данных временных рядов (TSDB), настройке политик хранения (Retention Policies) и синтаксису Flux-запросов.
- **Grafana** — Инструкция по созданию аналитических дашбордов, кастомизации панелей и настройке алертинга.
- **Слой автоматизации, брокерных шин и прикладных алгоритмов:**
- **Node-RED** — Руководство разработчика по созданию потоков автоматизации (flows), интеграции узлов и low-code программированию логических цепочек.
- **MQTT-Broker (Mosquitto)** — Инструкция по администрированию шины сообщений, настройке ACL-прав, топиков и конфигурации безопасности.
- **Telegraf** — Руководство по конфигурированию плагинов сбора данных, парсингу MQTT-топиков и оптимизации записи в InfluxDB.
- **Logic-Engine (Python)** — Техническое описание прикладных Python-скриптов, интеграции ML/AI-алгоритмов и математической обработки данных.
- **Служебный слой, шлюзование и мониторинг инфраструктуры:**
- **Apprise** — Инструкция по интеграции центра уведомлений, настройке шлюзов оповещений (Telegram, Email, SMS) и шаблонов сообщений.
- **Nginx-Auth** — Руководство по обеспечению безопасности периметра, веб-шлюзованию и управлению файлами авторизации .htpasswd.
- **Uptime Kuma** — Инструкция по созданию триггеров мониторинга доступности, настройке HTTP/TCP пингов и дашбордов состояния инфраструктуры.
- **Portainer** — Руководство администратора по визуальному управлению контейнерами, инспекции логов, Docker-сетей и персистентных томов.
  Все дополнительные руководства расположены в корневом каталоге проекта в папке ~/elis_vulcan-90/docs/.

1.3. Целевая аудитория

Настоящее руководство адресовано **широкому кругу специалистов и исследователей** — от промышленных инженеров до студентов профильных вузов. Документ написан так, чтобы быть **полезным как практику, так и новичку**: сложные концепции объясняются простым языком, а команды можно копировать напрямую.

1.3.1 Промышленные и системные администраторы

Для кого: специалисты, отвечающие за развертывание, эксплуатацию и отказоустойчивость промышленных SCADA-систем на предприятиях.

Что даёт документ:

- Полный цикл развертывания ELIS_Vulcan-90 на Ubuntu Server.
- Готовые скрипты автоматизации (install, uninstall, backup, restore).
- Методики резервного копирования и аварийного восстановления.
- Инструкции по обеспечению безопасности и обновлению.

1.3.2 Инженеры АСУ ТП и КИПиА

Для кого: инженеры, эксплуатирующие SCADA в реальном производстве — наладчики, технологи, специалисты по автоматизации.

Что даёт документ:

- Настройка подключения PLC по Modbus RTU через `/dev/PLC`.
- Работа с MQTT-шиной и интеграция с датчиками.
- Диагностика типовых неисправностей.
- Понимание архитектуры «от датчика до решений».

1.3.3 DevOps-инженеры и SRE

Для кого: специалисты, отвечающие за инфраструктуру, CI/CD, мониторинг и надежность сервисов.

Что даёт документ:

- Понимание микросервисной архитектуры из 12 контейнеров.
- Методики бэкапа Docker-образов (`docker save` / `docker load`).
- Перенос стека между серверами без интернета.
- Автоматизация через cron и systemd.
- Интеграция с Portainer, Uptime Kuma, Apprise.

1.3.4 Студенты профильных вузов

Для кого: студенты направлений «Автоматизация технологических процессов», «АСУ ТП», «Промышленная электроника», «Информационные системы и технологии», «Мехатроника и робототехника».

Что даёт документ:

- **Реальный опыт** работы с промышленной SCADA — построенной по канонам производства.
- **Понимание полного стека** технологий: Docker, MQTT, InfluxDB, Grafana, Modbus RTU, Node-RED.
- **Готовый проект для курсовой, дипломной или лабораторной работы.**
- **Возможность развернуть на одном ноутбуке** и экспериментировать без дорогостоящего оборудования.
- **Понятные команды** — даже без глубокого знания Linux можно пройти весь путь за короткое время.
- **Основа для научных публикаций** — сравнение SCADA-решений, исследование производительности, интеграция с ML-алгоритмами.

Особенно полезно для:

- Лабораторий по АСУ ТП и SCADA-системам.
- Курсов «Промышленный интернет вещей (IIoT)».
- Практикумов по Docker и микросервисам.
- Дипломных проектов по цифровизации производства.

1.3.5 Научные лаборатории и исследовательские центры

Для кого: научные сотрудники, аспиранты, инженеры-исследователи в области промышленной автоматизации, киберфизических систем, цифровых двойников, IIoT.

Что даёт документ:

- **Готовая платформа** для экспериментов — не нужно писать SCADA с нуля.
- **Модульная архитектура** — можно заменить любой компонент (например, FUXA на другую HMI, InfluxDB на TimescaleDB).
- **Открытый исходный код** — можно модифицировать под свои задачи.
- **Интеграция с Python-модулем** для ML-алгоритмов, прогнозирования, anomaly detection.
- **Сбор исторических данных** в InfluxDB для научного анализа.

- **Визуализация через Grafana** — готовые дашборды для публикаций.
- **Возможность масштабирования** — от одного PLC до целого цеха.
- **Совместимость с лабораторным оборудованием** — Arduino, ESP32, Raspberry Pi, промышленные PLC.

Направления исследований, где применима ELIS_Vulcan-90:

- Цифровые двойники технологических процессов.
- Предиктивное обслуживание оборудования (Predictive Maintenance).
- Обнаружение аномалий в промышленных данных.
- Оптимизация энергопотребления.
- Интеграция SCADA с AI/ML-моделями.
- Кибербезопасность промышленных систем.
- Edge-вычисления в промышленности.

1.3.6 Малый и средний бизнес

Для кого: небольшие производства, мастерские, лаборатории, стартапы, которые хотят **недорогую и открытую SCADA** без лицензионных отчислений.

Что даёт документ:

- Развертывание **за короткое время** на обычном ПК или мини-сервере.
- **Нулевая стоимость лицензий** — всё на open-source.
- **Готовая инфраструктура** — не нужно покупать дорогие промышленные решения.
- **Масштабирование** по мере роста производства.
- **Интеграция с Arduino/ESP32** — бюджетные контроллеры вместо дорогих PLC.

1.3.7 Open-source сообщество

Для кого: разработчики, контрибьюторы, энтузиасты, которые хотят **участвовать в развитии проекта** — добавлять новые сервисы, улучшать скрипты, писать документацию, переводить на другие языки.

Что даёт документ:

- **Понятную архитектуру** для быстрого старта.
- **Модульные скрипты** — легко расширять и модифицировать.
- **Четкие конвенции** — как вносить изменения.
- **Возможность форкнуть** и адаптировать под свои задачи.

1.3.8 Универсальность документа

Руководство написано по принципу «от простого к сложному»**:**

Уровень	Что нужно	Что делать
Новичок	Базовое знание Linux	Запустить ./install.sh и следовать инструкциям
Пользователь	Понимание Docker	Развернуть проект, настроить FUXA, создать мнемосхему
Администратор	Опыт работы с Linux, Docker	Настроить бэкапы, мониторинг, безопасность
Эксперт	Понимание архитектуры SCADA	Модифицировать, расширять, интегрировать с внешними системами

Ключевой принцип: **документ** не требует **предварительного опыта работы с SCADA.**

Достаточно базовых навыков Linux.

1.3.9 Миссия и ценности проекта

ELIS_Vulcan-90 — это не просто «ещё одна SCADA». Это образовательная платформа**,** инструмент для исследований **и** решение для реального производства **одновременно.**

Наша миссия

Сделать промышленную автоматизацию **доступной** — для студентов, малого бизнеса, научных лабораторий и энтузиастов. Без дорогих лицензий, без закрытых форматов, без вендор-лока.

Наши ценности

Ценности проекта отражены в его логотипе и философии:

Ценность	Что означает
Надежность	Система работает в реальных условиях производства, а не только в демонстрациях
Интеллект	Интеграция с ML/AI-алгоритмами, аналитика данных, прогнозирование
Гибкость	Модульная архитектура — можно заменить любой компонент
Простота	Установка за короткое время, понятные команды, подробная документация
Развитие	Open-source, постоянное улучшение, сообщество контрибьюторов

Почему это важно

Промышленная автоматизация традиционно была **закрытой и дорогой**:

- Коммерческие SCADA стоят **тысячи долларов** за лицензию.
- Обучение требует **дорогостоящего оборудования**.
- Студенты изучают теорию, но не работают с реальными системами.
- Малый бизнес не может позволить себе промышленные решения.

ELIS_Vulcan-90 меняет это. **Теперь:**

- Студент может развернуть полноценную SCADA на одном ноутбуке.
- Научная лаборатория получает готовую платформу для экспериментов.
- Малый бизнес внедряет автоматизацию без лицензионных отчислений.

- Open-source сообщество развивает проект совместно.

Что дальше:

Проект продолжает развиваться. В планах:

- **Расширение поддержки OPC UA** — FUXA уже поддерживает OPC UA «из коробки», поэтому интеграция с промышленным оборудованием возможна без доработок. В планах — готовые шаблоны мнемосхем для OPC UA-устройств и документация по настройке.
- **Интеграция с ML-фреймворками** (TensorFlow, PyTorch) — для предиктивной аналитики.
- **Мобильное приложение** для оператора.
- **Расширенная библиотека мнемосхем** — готовые шаблоны для типовых процессов.
- **Мультиязычная документация** — для международного сообщества.

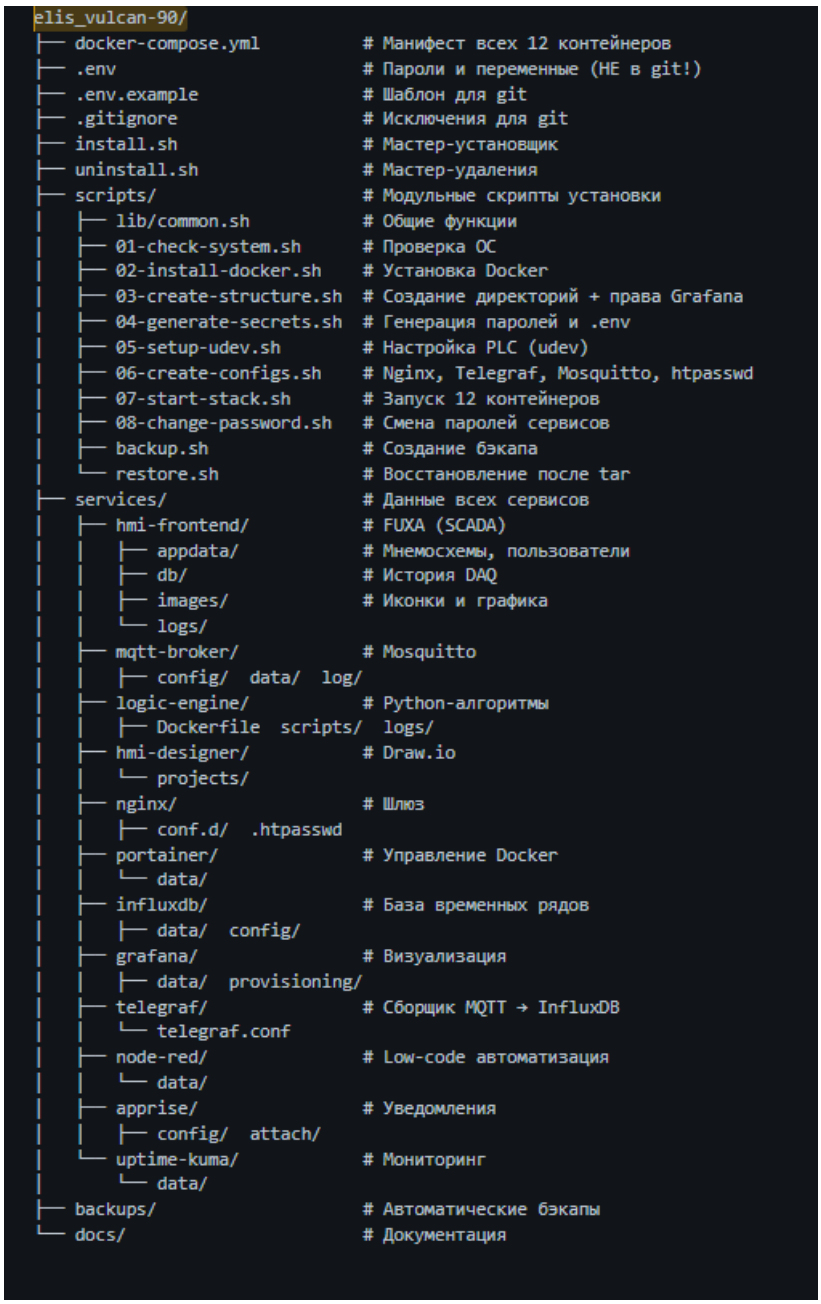
Присоединяйтесь к развитию ELIS_Vulcan-90!

Раздел 1. Архитектура и состав системы

1.1. Структура директорий проекта

Полная файловая структура корневой директории `elis_vulcan-90/`. Все подкаталоги внутри `services/` монтируются в соответствующие контейнеры и содержат как конфигурационные файлы, так и персистентные данные.

elis_vulcan-90/



Пояснения к ключевым каталогам:

Каталог	Содержимое	Критичность
`services/hmi-frontend/appdata/`	Мнемосхемы, настройки, пользователи FUXA	**P0 — критично**
`services/hmi-frontend/db/`	История DAQ-данных FUXA	**P0 — критично**
`services/hmi-frontend/`	Загруженные иконки и графика	**P0 — критично**

`images/`		
`services/influxdb/data/`	Вся история датчиков	**P0 — критично**
`services/grafana/data/`	Дашборды и настройки Grafana	**P0 — критично**
`services/logic-engine/scripts/`	Python-скрипты алгоритмов	P1
`services/node-red/data/`	Flows Node-RED	P1
`services/mqtt-broker/config/`	Конфигурация Mosquitto	P1
`services/nginx/`	Конфигурация шлюза, .htpasswd	P1
`.env`	Пароли, токены, переменные окружения	**P0 — критично**
`services/*/logs/`	Логи работы сервисов	P3 (можно терять)
`backups/`	Архивы резервных копий	Хранилище

1.2. Функциональные модули

Экосистема ELIS_Vulcan-90 включает 12 контейнеров, разделенных на 4 логических слоя.

Слой сбора данных и визуализации:

- hmi-frontend (FUXA) — ядро SCADA, отображение мнемосхем, опрос PLC по Modbus RTU через /dev/PLC.

- mqtt-broker (Mosquitto) — центральная шина обмена сообщениями MQTT.

- logic-engine (Python) — модуль прикладных алгоритмов, обработка данных с датчиков.

Слой хранения и аналитики:

- influxdb — база данных временных рядов (TSDB) для хранения истории.

- telegraf — сборщик данных MQTT → InfluxDB, работающий независимо от FUXA.

- grafana — визуализация трендов, дашборды, отчёты.

Слой автоматизации и уведомлений:

- node-red — low-code платформа для построения логических цепочек.

- apprise — центр уведомлений (Telegram, Email, SMS и др.).

Служебный слой:

- hmi-designer (Draw.io) — редактор схем.

- nginx-auth — защитный шлюз с базовой авторизацией для Draw.io.

- portainer — веб-интерфейс управления Docker.

- uptime-kuma — мониторинг доступности всех сервисов.

1.3. Карта портов

Порт	Сервис	Назначение
`1881`	hmi-frontend	Веб-интерфейс FUXA (SCADA)
`1883`	mqtt-broker	MQTT TCP
`9001`	mqtt-broker	MQTT WebSocket
`8080`	nginx-auth	Draw.io (защищён паролем)
`8086`	influxdb	Веб-интерфейс InfluxDB
`3000`	grafana	Дашборды Grafana
`1880`	node-red	Low-code автоматизация
`8000`	apprise	API и веб-интерфейс Apprise
`3001`	uptime-kuma	Мониторинг доступности
`9000`, `9443`	portainer	Управление Docker

Конфигурации паролей и токенов вынесены в файл .env в корне проекта. Файл .env **не должен попадать в системы контроля версий и передаваться по открытым каналам.**

⚠️ *Файл `.env` не должен попадать в git и передаваться по открытым каналам.*

1.4. Критичность данных

(раздел продублирован для удобства навигации)

Каталог	Содержимое	Критичность
`services/hmi-frontend/appdata/`	Мнемосхемы, настройки, пользователи FUXA	**P0 — критично**
`services/hmi-frontend/db/`	История DAQ-данных FUXA	**P0 — критично**
`services/hmi-frontend/images/`	Загруженные иконки и графика	**P0 — критично**
`services/influxdb/data/`	Вся история датчиков	**P0 — критично**
`services/grafana/data/`	Дашборды и настройки Grafana	**P0 — критично**
`services/logic-engine/scripts/`	Python-скрипты алгоритмов	P1
`services/node-red/data/`	Flows Node-RED	P1
`services/mqtt-broker/config/`	Конфигурация Mosquitto	P1
`services/nginx/`	Конфигурация шлюза, .htpasswd	P1
`.env`	Пароли, токены, переменные окружения	**P0 — критично**
`services/*/logs/`	Логи работы сервисов	P3 (можно терять)
`backups/`	Архивы резервных копий	Хранилище

Раздел 2. Требования и подготовка сервера

Прежде чем развертывать ELIS_Vulcan-90, убедитесь, что сервер соответствует требованиям, и выполните подготовительные шаги. Этот раздел — **фундамент успешной установки**.

2.1. Аппаратные требования

Минимальная конфигурация (для теста и разработки)

Компонент	Минимум	Почему
CPU	2 ядра	12 контейнеров + InfluxDB + Grafana. 2 ядра — **только для проверки**, не для реальной работы.
RAM	8 ГБ	4 ГБ — **не хватит**. InfluxDB просит 4 ГБ , Grafana — 2–4 ГБ , остальные сервисы — ещё 2–3 ГБ.
Диск	60 ГБ (HDD)	Образы Docker (~4 ГБ) + данные InfluxDB + логи + бэкапы. 40 ГБ — **впритык**, быстро

		закончится.
Сеть	100 Мбит/с	Статический IP обязателен.
USB	1 порт	Для PLC (Arduino/CH340/FTDI).

Рекомендованная конфигурация (для production)

Компонент	Рекомендовано	Почему
CPU	4 ядра	Grafana для «medium» tier требует 4–8 ядер
RAM	16 ГБ	Grafana medium — 8–16 ГБ . Node-RED с базами — 16 ГБ . InfluxDB — 4+ ГБ
Диск	120 ГБ (SSD)	SSD критичен для InfluxDB — HDD даст медленные запросы. 120 ГБ с запасом на историю и бэкапы.
Сеть	1 Гбит/с	Для быстрой синхронизации с Grafana и InfluxDB.
USB	2+ порта	PLC + возможно USB-накопитель для бэкапов.

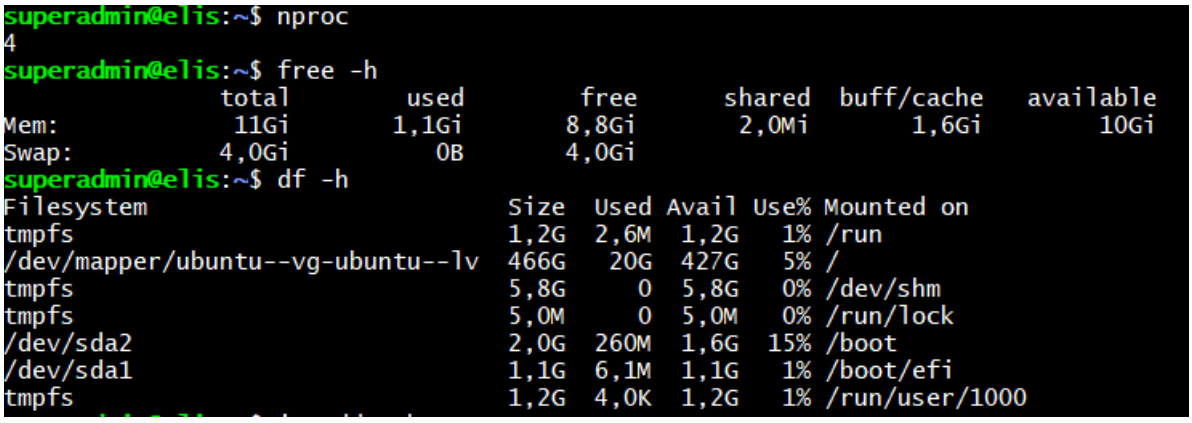
Проверка ресурсов на сервере

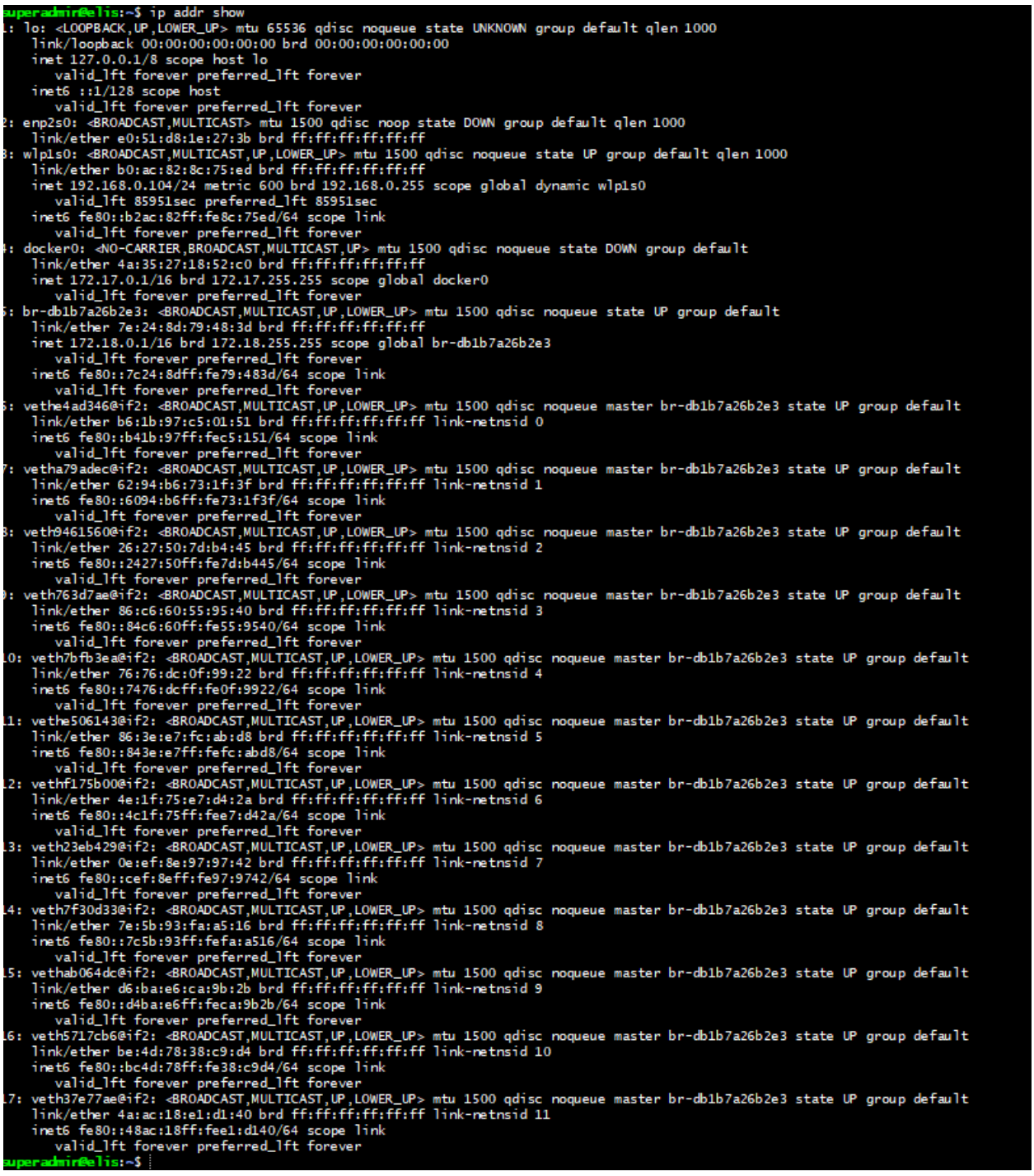
```
# CPU
nproc

# RAM
free -h

# Диск
df -h /

# Сеть
ip addr show
```





2.2. Программные требования

Операционная система:

- **Ubuntu Server 22.04 LTS** (проверено) или новее.
- **Debian 12** (должно работать, но не тестировалось).
- Другие дистрибутивы — на свой риск.

Обязательные пакеты:

Пакет	Назначение	Установка
`docker-ce`	Docker Engine	`curl -fsSL https://get.docker.com | sh`
`docker-compose-plugin`	Docker Compose v2	Идёт с Docker
`curl`	Скачивание	`sudo apt install curl`
`tar`, `gzip`	Архивы	`sudo apt install tar gzip`
`openssl`	Генерация паролей	`sudo apt install openssl`
`apache2-utils`	`htpasswd` для Draw.io	`sudo apt install apache2-utils`

Проверка версии ОС:

```
bash
lsb_release -a
# или
cat /etc/os-release
```

Ожидается:

```
superadmin@elis:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.5 LTS
Release:        22.04
Codename:       jammy
superadmin@elis:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
superadmin@elis:~$
```

2.3. Подготовка Ubuntu Server

2.3.1. Обновление системы

```
bash
sudo apt update
sudo apt upgrade -y
sudo apt autoremove -y
```

2.3.2. Настройка часового пояса

```
bash
sudo timedatectl set-timezone Europe/Moscow
timedatectl
```

Ожидается: `Time zone: Europe/Moscow (MSK, +0300)`.

2.3.3. Установка базовых утилит

```
bash
sudo apt install -y \
    curl \
    wget \
    git \
    nano \
    htop \
    tree \
    net-tools \
    openssl \
    apache2-utils \
    tar \
    gzip \
    rsync
```

2.3.4. Создание пользователя (если нужно)

Если работаете от root — создайте отдельного пользователя:

```
bash
sudo adduser superadmin
sudo usermod -aG sudo superadmin
su - superadmin
```

Дальнейшие шаги выполняются от имени этого пользователя.

2.3.5. Проверка sudo

```
bash
```

```
sudo -v
```

Если запрашивает пароль — значит настройки применились.

2.4. Настройка сети и статического IP

2.4.1. Определение текущего IP

```
bash
ip addr show
# или
hostname -I
```

2.4.2. Настройка статического IP через Netplan

Ubuntu Server 22.04 использует **Netplan**. Найдите файл конфигурации:

```
bash
ls /etc/netplan/
# Обычно: 00-installer-config.yaml или 50-cloud-init.yaml
```

Откройте его:

```
bash
sudo nano /etc/netplan/00-installer-config.yaml
```

Пример конфигурации для статического IP `192.168.0.100`**:**

```
yaml
network:
  version: 2
  ethernets:
    ens33:                          # ← замените на ваше имя интерфейса
      dhcp4: no
      addresses:
        - 192.168.0.100/24
      routes:
        - to: default
          via: 192.168.0.1
```

```
      nameservers:
        addresses:
          - 8.8.8.8
          - 1.1.1.1
```

Примените:

```
bash
sudo netplan apply
```

Проверьте:

```
bash
ip addr show
ping -c 3 8.8.8.8
```

2.4.3. Настройка hosts (опционально)

Для удобства добавьте в /etc/hosts:

```
bash
sudo nano /etc/hosts
```

Добавьте:

```
text
192.168.0.100   elis-vulcan-90
```

Теперь можно обращаться по имени `elis-vulcan-90` вместо IP.

2.4.4. Настройка firewall (опционально)

⚠️ *Не открывайте наружу порты без необходимости! Для внутренней сети:*

```
bash
sudo ufw allow from 192.168.0.0/24 to any port 1881   # FUXA
sudo ufw allow from 192.168.0.0/24 to any port 8080   # Draw.io
sudo ufw allow from 192.168.0.0/24 to any port 3000   # Grafana
sudo ufw enable
sudo ufw status
```

Для продакшена — используйте VPN (WireGuard) вместо открытых портов.

2.5. Проверка готовности к установке

Перед запуском install.sh выполните **финальную проверку**:

```
bash
echo "═══ 1. ОС ═══"
lsb_release -d

echo "═══ 2. Архитектура ═══"
uname -m

echo "═══ 3. Свободное место ═══"
df -h / | tail -1

echo "═══ 4. RAM ═══"
free -h | grep Mem

echo "═══ 5. CPU ═══"
nproc

echo "═══ 6. sudo ═══"
sudo -v && echo "OK"

echo "═══ 7. Утилиты ═══"
for cmd in curl tar gzip sudo openssl; do
    command -v "$cmd" >/dev/null && echo "  ✓ $cmd" || echo "  ✗ $cmd — ОТСУТСТВУЕТ"
done

echo "═══ 8. Docker ═══"
docker --version 2>/dev/null || echo "  Docker не установлен — установится на этапе 02"
```

Если все проверки прошли успешно — переходите к разделу 3.

РАЗДЕЛ 3. ПЕРВИЧНАЯ УСТАНОВКА

Этот раздел описывает **ручную установку** ELIS_Vulcan-90 — шаг за шагом, без автоматизации. Он нужен, чтобы вы **понимали, что происходит**, прежде чем запускать install.sh. Если хотите сразу установить — переходите к **Разделу 4**.

3.1. Установка Docker и Docker Compose

Установите Docker Engine из официального репозитория и добавьте текущего пользователя в группу docker:

3.1.1. Установка Docker

```bash
curl -fsSL https://get.docker.com | sudo sh
```

Скрипт установит:

- docker-ce — Docker Engine.
- docker-ce-cli — CLI.
- containerd.io — runtime.
- docker-compose-plugin — Compose v2.
- docker-buildx-plugin — Buildx.

3.1.2. Добавление пользователя в группу docker

```bash
sudo usermod -aG docker $USER
newgrp docker
```

Проверка:

```bash
groups | grep docker
# Должно быть: ... docker ...
```

⚠️ *Если группа не появилась — перелогиньтесь (exit + ssh).*

3.1.3. Проверка версий

```bash
docker --version
docker compose version
```

Ожидается:

```
text
Docker version 29.x.x, build ...
Docker Compose version v5.x.x
```

3.1.4. Проверка работоспособности

```
bash
docker ps
```

Ожидается: пустая таблица с заголовками — это нормально, контейнеров пока нет.

⚠️ ***Если `permission denied` — перелогиньтесь или `newgrp docker`.***

3.2. Развертывание проекта из архива

3.2.1. Распаковка данных

```
bash
cd ~
tar -xzf elis-vulcan-90-data-*.tar.gz
```

Проверка:

```
bash
ls -la ~/elis_vulcan-90/
# Ожидается: docker-compose.yml, install.sh, uninstall.sh, scripts/,
services/ и т.д.
```

3.2.2. Загрузка Docker-образов

```
bash
docker load < elis-vulcan-90-images-*.tar.gz
```

Время: 1–3 минуты (архив ~1.8 ГБ).

Проверка:

```
bash
docker images | grep elis-vulcan-90
# Ожидается: elis_vulcan-90-logic-engine
```

```
docker images | grep -cE "frangoteam|eclipse|influx|grafana|telegraf|node-red|apprise|uptime|portainer|drawio|nginx"
# Ожидается: 11
```

Если образов меньше 12 — проверьте, что архив полный:

```
bash
tar -tzf elis-vulcan-90-images-*.tar.gz 2>&1 | grep -i "cannot\|permission" || echo "Ошибок нет"
```

3.3. Создание файла .env

3.3.1. Создание корневого .env

Создайте ~/elis_vulcan-90/.env:

```
bash
nano ~/elis_vulcan-90/.env
```

Содержимое:

```
bash
# ELIS_Vulcan-90 — корневой .env
TZ=Europe/Moscow

# Версии образов
FUXA_TAG=latest
MOSQUITTO_TAG=latest
INFLUXDB_TAG=2.7-alpine
GRAFANA_TAG=latest
TELEGRAF_TAG=latest
NODE_RED_TAG=latest
APPRISE_TAG=latest
UPTIME_KUMA_TAG=latest
PORTAINER_TAG=latest
DRAWIO_TAG=latest
NGINX_TAG=alpine
```

3.3.2. Создание .env для каждого сервиса

InfluxDB — `services/influxdb/.env`:

```
bash
```

```
TZ=Europe/Moscow
DOCKER_INFLUXDB_INIT_MODE=setup
DOCKER_INFLUXDB_INIT_USERNAME=admin
DOCKER_INFLUXDB_INIT_PASSWORD=<пароль_12+_символов>
DOCKER_INFLUXDB_INIT_ORG=elis_vulcan
DOCKER_INFLUXDB_INIT_BUCKET=scada_data
DOCKER_INFLUXDB_INIT_RETENTION=30d
```

Grafana — `services/grafana/.env`:

```
bash
TZ=Europe/Moscow
GF_SECURITY_ADMIN_USER=admin
GF_SECURITY_ADMIN_PASSWORD=<пароль_12+_символов>
```

Telegraf — `services/telegraf/.env`:

```
bash
TZ=Europe/Moscow
INFLUXDB_URL=http://influxdb:8086
INFLUXDB_TOKEN=
INFLUXDB_ORG=elis_vulcan
INFLUXDB_BUCKET=scada_data
```

⚠️ *INFLUXDB_TOKEN* ***заполняется позже — после первого запуска InfluxDB (см. раздел 12.4).***

Остальные сервисы — только TZ:

```
bash
for svc in hmi-frontend mqtt-broker hmi-designer nginx node-red apprise
uptime-kuma portainer; do
    echo "TZ=Europe/Moscow" > ~/elis_vulcan-90/services/${svc}/.env
done
```

logic-engine — services/logic-engine/.env:

```
bash
TZ=Europe/Moscow
MQTT_BROKER=mqtt-broker
MQTT_PORT=1883
```

3.3.3. Права на .env

```bash
chmod 600 ~/elis_vulcan-90/.env
chmod 600 ~/elis_vulcan-90/services/*/.env
```

3.3.4. Обновление прав Grafana

```bash
sudo chown -R 472:472 ~/elis_vulcan-90/services/grafana/data/
sudo chown -R 1000:1000 ~/elis_vulcan-90/services/influxdb/data/
```

3.4. Настройка udev для PLC

3.4.1. Подключение PLC

Подключите PLC к USB-порту сервера. Определите VID:PID:

```bash
lsusb
```

Пример вывода:

```text
Bus 001 Device 003: ID 2341:0043 Arduino SA Uno R3 (CDC ACM)
```

Здесь 2341:0043 — VID:PID.

3.4.2. Создание udev-правила

```bash
sudo nano /etc/udev/rules.d/99-plc.rules
```

Содержимое:

```text
# ELIS_Vulcan-90 — правила для PLC
# Создаёт стабильный симлинк /dev/arduino
```

```
# Arduino Uno (оригинал, ATmega16U2)
SUBSYSTEM=="tty", ATTRS{idVendor}=="2341", ATTRS{idProduct}=="0043",
SYMLINK+="arduino", MODE="0660", GROUP="dialout"

# Клоны на CH340
SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523",
SYMLINK+="arduino", MODE="0660", GROUP="dialout"

# Клоны на CP2102
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60",
SYMLINK+="arduino", MODE="0660", GROUP="dialout"

# Промышленные PLC на FTDI
SUBSYSTEM=="tty", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001",
SYMLINK+="arduino", MODE="0660", GROUP="dialout"
```

3.4.3. Применение правила

```bash
sudo udevadm control --reload-rules
sudo udevadm trigger

# Добавить пользователя в группу dialout
sudo usermod -aG dialout $USER
```

⚠️ *Требуется перелогин для применения группы dialout.*

3.4.4. Проверка

```bash
ls -l /dev/arduino
# Ожидается: crw-rw---- 1 root dialout ... /dev/arduino

ls -l /dev/ttyACM0
# crw-rw---- 1 root dialout ... /dev/ttyACM0
```

Если `/dev/arduino` нет — PLC не подключён или правило не применилось.

⚠️ **Если PLC пока нет — временно закомментируйте `devices:` в `docker-compose.yml` для `hmi-frontend`.**

3.5. Первичный запуск стека

3.5.1. Запуск

```bash
cd ~/elis_vulcan-90
docker compose up -d
```

Время: 30–60 секунд (образы уже загружены).

Ожидаемый вывод:

```text
[+] up 13/13
 ✔️ Network elis_vulcan-90_default   Created
 ✔️ Container mqtt_broker            Started
 ✔️ Container influxdb               Started
 ✔️ Container grafana                Started
 ✔️ Container hmi_frontend           Started
```

3.5.2. Проверка статуса

```bash
docker ps
```

```
superadmin@elis:~$ docker ps
CONTAINER ID   IMAGE                           COMMAND                  CREATED      STATUS                    PORTS                                                
```

3.5.4. Если что-то не Up

```bash
docker compose logs <service> --tail=30
```

Частые причины:

Симптом	Решение
Grafana `Restarting`	`sudo chown -R 472:472 services/grafana/data/`
Mosquitto `Restarting`	Проверить `services/mqtt-broker/config/mosquitto.conf`
InfluxDB пустой	Проверить `services/influxdb/data/` и `.env`
FUXA упал	`docker compose logs hmi-frontend`

РАЗДЕЛ 4. АВТОМАТИЧЕСКАЯ УСТАНОВКА ЧЕРЕЗ СКРИПТЫ

ELIS_Vulcan-90 имеет **модульный установщик** из 7 скриптов. Каждый скрипт отвечает за **один этап** и может быть запущен **отдельно** — это удобно для отладки.

Преимущества автоматизации:

- **Идемпотентность** — можно запускать повторно без вреда.
- **Проверка ошибок** — при падении этап останавливается, показывает логи.
- **Гибкость** — флаги для пропуска этапов.
- **Логирование** — всё пишется в /tmp/elis-vulcan-*.log.

4.1. Полная установка одной командой

```bash
cd ~/elis_vulcan-90
./install.sh
```

Что произойдёт:

install.sh последовательно вызовет скрипты 01–07:

1. **01-check-system.sh** — проверка ОС, sudo, утилит, места.
2. **02-install-docker.sh** — установка Docker.
3. **03-create-structure.sh** — создание директорий + chown 472:472.
4. **04-generate-secrets.sh** — генерация паролей и .env.

5. **05-setup-udev.sh** — udev-правило для PLC.
6. **06-create-configs.sh** — Nginx, Telegraf, Mosquitto, htpasswd.
7. **07-start-stack.sh** — `docker compose up -d` + умная проверка.

⚠️ *После этапа 02 требуется перелогин — но **install.sh** об этом предупредит.*

Ожидаемый вывод в конце:

```
text
▶️ Этап 07: Запуск стека
[ OK ] Все 12 контейнеров успешно запущены!

═══════════════════════════════════════════════
  ELIS_Vulcan-90 — Установка завершена!
═══════════════════════════════════════════════
```

Время полной установки: 5–15 минут (первый раз — дольше, потому что скачиваются образы).

4.2. Пошаговая установка (для отладки)

Если хотите контролировать каждый этап:

```
bash
cd ~/elis_vulcan-90

# Этап 01 — проверка системы
./scripts/01-check-system.sh

# Этап 02 — установка Docker
./scripts/02-install-docker.sh

# ⚠️ ПЕРЕЛОГИН (logout/login) для применения группы docker

# Этап 03 — создание структуры директорий + права Grafana
./scripts/03-create-structure.sh

# Этап 04 — генерация паролей и .env
./scripts/04-generate-secrets.sh
# ⚠️ СОХРАНИТЕ ПАРОЛИ из вывода!

# Этап 05 — настройка udev для PLC
```

```
./scripts/05-setup-udev.sh

# Этап 06 — создание конфигов
./scripts/06-create-configs.sh

# Этап 07 — запуск 12 контейнеров
./scripts/07-start-stack.sh
```

Преимущества пошагового запуска:

- Видите результат каждого этапа.
- Можете остановиться и проверить.
- Легче найти проблему.

4.3. Флаги установщика

Флаг	Что делает
`--skip-docker`	Пропустить установку Docker (если уже есть)
`--skip-udev`	Не настраивать udev (если нет PLC)
`--skip-start`	Только подготовка, без запуска
`--only NN [NN]`	Выполнить только указанные этапы
`--dir PATH`	Установить в другую директорию
`--help`	Показать справку

Примеры:

```
bash
# Только этапы 03 и 04
./install.sh --only 03 04

# Пропустить установку Docker и udev
./install.sh --skip-docker --skip-udev

# Установить в /opt/elis-vulcan
./install.sh --dir /opt/elis-vulcan

# Только подготовка, без запуска
./install.sh --skip-start
```

4.4. Что делает каждый скрипт

Скрипт	Что делает	Идемпотентен?

01-check-system.sh	Проверка ОС, архитектуры, sudo, утилит, места	Да
02-install-docker.sh	Установка Docker + добавление в группу	Да
03-create-structure.sh	Создание директорий + chown 472:472 для Grafana	Да
04-generate-secrets.sh	Генерация паролей, .env, .secrets-*.txt	⚠️ --force для перезаписи
05-setup-udev.sh	udev-правило + добавление в dialout	Да
06-create-configs.sh	Nginx, Telegraf, Mosquitto, htpasswd, Dockerfile	Да
07-start-stack.sh	docker compose up -d + умная проверка	Да

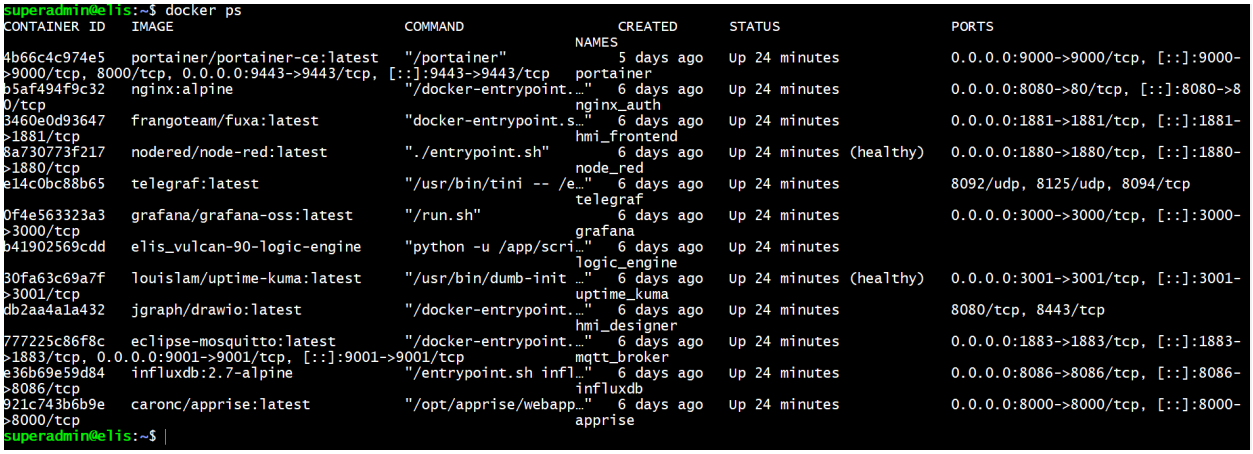
Что значит «идемпотентен»: **можно запускать повторно — ничего не сломается.**

4.5. Проверка после установки

4.5.1. Все контейнеры Up?

```
bash
cd ~/elis_vulcan-90
docker ps
```

Ожидается: 12 из 12 в Up.



4.5.2. HTTP-проверка

```
bash
curl -s -o /dev/null -w "FUXA:     HTTP %{http_code}\n" http://localhost:1881
curl -s -o /dev/null -w "Grafana:  HTTP %{http_code}\n"
http://localhost:3000/api/health
curl -s -o /dev/null -w "InfluxDB: HTTP %{http_code}\n"
http://localhost:8086/health
```

```
curl -s -o /dev/null -w "Draw.io:  HTTP %{http_code}\n" http://localhost:8080
curl -s -o /dev/null -w "Node-RED: HTTP %{http_code}\n" http://localhost:1880
curl -s -o /dev/null -w "Apprise:  HTTP %{http_code}\n" http://localhost:8000
curl -s -o /dev/null -w "Uptime:   HTTP %{http_code}\n" http://localhost:3001
```

Ожидаемые коды:

Сервис	Код
FUXA	200
Grafana	200
InfluxDB	200
Draw.io	401
Node-RED	200
Apprise	200
Uptime Kuma	302

4.5.3. Все .env на месте?

```
bash
find ~/elis_vulcan-90/services -maxdepth 2 -name ".env" -type f 2>/dev/null | wc -l
# Ожидается: 12
```

4.5.4. Проверка паролей

```
cat ~/elis_vulcan-90/.secrets-*.txt
```

⚠️ *Сохраните пароли в менеджер и удалите файл:*

```
bash
shred -u ~/elis_vulcan-90/.secrets-*.txt
```

4.6. Что дальше

После успешной установки:

1. **Открыть FUXA** — http://<IP>:1881.
2. **Настроить MQTT-устройство** — Server = mqtt-broker, Port = 1883.
3. **Получить API-токен InfluxDB** — http://<IP>:8086.
4. **Вписать токен в Telegraf** — services/telegraf/.env.
5. **Настроить Grafana** — источник данных InfluxDB.
6. **Создать администратора Portainer** — https://<IP>:9443.

7. **Создать администратора Uptime Kuma** — http://<IP>:3001.

Детали — в соответствующих разделах документа.

РАЗДЕЛ 5. УПРАВЛЕНИЕ ЖИЗНЕННЫМ ЦИКЛОМ КОНТЕЙНЕРОВ

5.1. Основные команды

Операция	Команда
Запуск всего стека	`docker compose up -d`
Остановка	`docker compose down`
Перезапуск одного сервиса	`docker compose restart hmi-frontend`
Просмотр логов	`docker compose logs -f --tail=100 <service>`
Проверка статуса	`docker ps`
Использование ресурсов	`docker stats`

5.2. Изоляция контейнеров

Каждый контейнер работает в собственной сети Docker Compose и обращается к другим сервисам **по имени service, а не по container_name**. Это критически важно: в настройках FUXA MQTT-устройство должно указывать mqtt-broker, а не mosquitto или mqtt_broker.

5.3. Работа с томами

Все данные монтируются bind-mount'ом на хост. Удаление директорий в services/ приведёт к **необратимой потере данных**. Перед любыми манипуляциями с каталогами обязательно делайте резервную копию.

РАЗДЕЛ 6. РЕЗЕРВНОЕ КОПИРОВАНИЕ

6.1. Критически важные данные

Приоритет P0 (без них система теряет смысл):

- services/hmi-frontend/appdata/project.fuxap.db — мнемосхемы FUXA.
- services/hmi-frontend/db/ — исторические данные DAQ.
- services/hmi-frontend/images/ — загруженные иконки.
- services/influxdb/data/ — вся история датчиков.
- services/grafana/data/ — дашборды.
- .env, services/*/.env — пароли и токены.

Приоритет P1:

- services/logic-engine/scripts/ — Python-алгоритмы.
- services/node-red/data/ — flows Node-RED.
- services/mqtt-broker/config/ — конфиг брокера.
- services/nginx/ — конфиг шлюза.
- docker-compose.yml.

6.2. Ручное создание полного бэкапа

⚠️ *Обязательно использовать* ***sudo*** ***— иначе не сохранятся файлы, принадлежащие root и внутренним UID (Grafana, Mosquitto, Portainer).***

```
bash
cd ~/elis_vulcan-90

# 1. Остановить FUXA и InfluxDB для консистентности БД
docker compose stop hmi-frontend influxdb

# 2. Создать архив с sudo (сохраняет ВСЕ файлы)
sudo tar -czf ~/elis-vulcan-FULL-BACKUP-$(date +%Y%m%d-%H%M).tar.gz -C ~
elis_vulcan-90/
```

```
# 3. Вернуть владельца
sudo chown $USER:$USER ~/elis-vulcan-FULL-BACKUP-*.tar.gz

# 4. Запустить обратно
docker compose start hmi-frontend influxdb

# 5. Проверить размер (~100–500 МБ)
ls -lh ~/elis-vulcan-FULL-BACKUP-*.tar.gz

# 6. Проверить, что нет ошибок
tar -tzf ~/elis-vulcan-FULL-BACKUP-*.tar.gz 2>&1 | grep -iE "cannot|
permission" || echo "Ошибок нет"
```

Что должно быть в архиве (проверка):

```
tar -tzf ~/elis-vulcan-FULL-BACKUP-*.tar.gz 2>/dev/null | grep -E
"project.fuxap.db|users.fuxap.db|grafana.db|influxd.bolt"
```

Ожидаемый вывод:

```
text
elis_vulcan-90/services/hmi-frontend/appdata/project.fuxap.db
elis_vulcan-90/services/hmi-frontend/appdata/users.fuxap.db
elis_vulcan-90/services/grafana/data/grafana.db
elis_vulcan-90/services/influxdb/data/influxd.bolt
```

6.3. Создание образов всех контейнеров (docker save)

Этот раздел — ключевой для переноса на другой сервер без интернета**. Он создаёт «золотой слепок» всех Docker-образов.**

6.3.1. Что сохраняется

Все 12 Docker-образов:

- frangoteam/fuxa:latest

- eclipse-mosquitto:latest
- influxdb:2.7-alpine
- grafana/grafana-oss:latest
- telegraf:latest
- nodered/node-red:latest
- caronc/apprise:latest
- louislam/uptime-kuma:latest
- portainer/portainer-ce:latest
- jgraph/drawio:latest
- nginx:alpine
- elis_vulcan-90-logic-engine:latest (образ алгоритмов)

6.3.2. Создание архива образов

⚠️ ***Обязательно через*** *docker compose config --images — он возвращает* ***точные*** *имена, включая elis_vulcan-90-logic-engine (с подчёркиванием, не дефисом!).*

```
bash
cd ~/elis_vulcan-90
mkdir -p ~/elis-vulcan-backups

# Создать архив всех образов
docker save $(docker compose config --images | sort -u) \
    | gzip > ~/elis-vulcan-backups/full-stack-$(date +%F).tar.gz

# Проверить размер (~1.8–2 ГБ)
ls -lh ~/elis-vulcan-backups/
```

Ожидаемый вывод:

```
text
-rw-rw-r-- 1 superadmin superadmin 1,8G ... full-stack-2026-09-25.tar.gz
```

Время создания: 1–3 минуты. **Не прерывайте!**

6.3.3. Проверка архива образов

```
bash
# Проверить, что архив валидный
```

```
tar -tzf ~/elis-vulcan-backups/full-stack-*.tar.gz 2>/dev/null | head -5
```

Более точная проверка — загрузить обратно в тестовом режиме (опционально):

```
bash
# Проверить, что Docker может прочитать архив (без реальной загрузки)
gunzip -c ~/elis-vulcan-backups/full-stack-*.tar.gz | head -c 100 | file -
# Ожидается: "POSIX tar archive"
```

6.3.4. ⚠️ Важные нюансы `docker save`

Проблема	Решение
`No such image: elis-vulcan-90-logic-engine`	Имя с дефисом, а нужно с подчёркиванием: elis_vulcan-90-logic-engine
docker save падает на середине	Один из образов отсутствует. Проверьте: docker compose config --images
Архив весит 20 байт	2>/dev/null скрыл ошибку. Запустите без него
Размер архива > 5 ГБ	Уберите latest из тегов, используйте конкретные версии

6.4. Автоматический бэкап через cron

Создайте `/etc/cron.daily/elis-vulcan-backup`:

```
bash
sudo tee /etc/cron.daily/elis-vulcan-backup <<'EOF'
#!/bin/bash
BACKUP_DIR=/home/superadmin/backups/elis-vulcan-90
mkdir -p "$BACKUP_DIR"

# Остановить FUXA и InfluxDB для консистентности
docker compose -f /home/superadmin/elis_vulcan-90/docker-compose.yml \
    stop hmi-frontend influxdb

# Создать архив
sudo tar -czf "$BACKUP_DIR/backup-$(date +%F).tar.gz" \
    -C /home/superadmin \
    elis_vulcan-90/
```

```
sudo chown superadmin:superadmin "$BACKUP_DIR/backup-$(date +%F).tar.gz"

# Запустить обратно
docker compose -f /home/superadmin/elis_vulcan-90/docker-compose.yml \
    start hmi-frontend influxdb

# Удалить архивы старше 30 дней
find "$BACKUP_DIR" -name "backup-*.tar.gz" -mtime +30 -delete
EOF

sudo chmod +x /etc/cron.daily/elis-vulcan-backup
```

Замените superadmin **на ваше имя пользователя**.

6.5. Проверка целостности бэкапа

Регулярно (раз в месяц) **проверяйте, что бэкапы можно восстановить:**

```
bash
# 1. Посмотреть содержимое
tar -tzf ~/elis-vulcan-FULL-BACKUP-*.tar.gz | head -20

# 2. Проверить ключевые файлы
tar -tzf ~/elis-vulcan-FULL-BACKUP-*.tar.gz | grep -E
"project.fuxap.db|.env$"

# 3. Распаковать в /tmp и проверить
mkdir -p /tmp/test-restore
sudo tar -xzf ~/elis-vulcan-FULL-BACKUP-*.tar.gz -C /tmp/test-restore
ls -la /tmp/test-restore/elis_vulcan-90/
sudo rm -rf /tmp/test-restore
```

6.6. Ротация бэкапов

Тип	Где хранить	Срок хранения	Частота
Полный бэкап (tar)	`~/elis-vulcan-backups/`	30 дней	Ежедневно (cron)
Архив образов	`~/elis-vulcan-backups/`	60 дней	При обновлении
«Золотой слепок»	Внешний диск / облако	Постоянно	Раз в квартал

Скрипт ротации (добавить в cron):

```
bash
# Удалить полные бэкапы старше 30 дней
find ~/elis-vulcan-backups -name "backup-*.tar.gz" -mtime +30 -delete

# Удалить архивы образов старше 60 дней
find ~/elis-vulcan-backups -name "full-stack-*.tar.gz" -mtime +60 -delete

# Удалить логи установки старше 7 дней
find /tmp -name "elis-vulcan-*.log" -mtime +7 -delete
```

6.7. Бэкап «золотого слепка» образов

После каждого **успешного обновления стека:**

```
bash
cd ~/elis_vulcan-90

# Создать новый «золотой слепок»
docker save $(docker compose config --images | sort -u) \
    | gzip > ~/elis-vulcan-backups/full-stack-$(date +%F).tar.gz

# Сохранить в долговременное хранилище (внешний диск, S3, Nextcloud)
rsync -av ~/elis-vulcan-backups/full-stack-*.tar.gz /mnt/external/backups/

# Или в S3 (если установлен aws-cli)
aws s3 cp ~/elis-vulcan-backups/full-stack-*.tar.gz s3://your-bucket/elis-vulcan/
```

РАЗДЕЛ 7. ВОССТАНОВЛЕНИЕ СИСТЕМЫ

7.1. Восстановление из полного бэкапа

```
bash
# 1. Остановить стек
cd ~/elis_vulcan-90
docker compose down

# 2. Удалить текущую директорию (опционально)
cd ~
sudo rm -rf elis_vulcan-90/

# 3. Распаковать архив
sudo tar -xzf ~/elis-vulcan-FULL-BACKUP-*.tar.gz -C ~
sudo chown -R $USER:$USER ~/elis_vulcan-90/

# 4. Перейти и дать права на скрипты
cd ~/elis_vulcan-90
chmod +x install.sh uninstall.sh scripts/*.sh scripts/lib/*.sh

# 5. КРИТИЧНО: Восстановить права сервисов (UID внутри контейнеров)
sudo chown -R 472:472 services/grafana/data/
sudo chown -R 1000:1000 services/influxdb/data/
sudo chown -R 1883:1883 services/mqtt-broker/data/
sudo chown -R 1883:1883 services/mqtt-broker/log/
sudo chown -R root:root services/portainer/data/
sudo chown -R root:root services/uptime-kuma/data/
sudo chown -R 1000:1000 services/node-red/data/

# 6. Запустить стек
docker compose up -d
sleep 25
docker compose ps
```

⚠️ *Без шага 5 Grafana и Mosquitto упадут с* ***`Permission denied`****!*

7.2. Скрипт `restore.sh` — автоматизация прав

Создайте `~/elis_vulcan-90/scripts/restore.sh`:

```
bash
#!/usr/bin/env bash
# ELIS_Vulcan-90 — Восстановление прав после tar
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "$SCRIPT_DIR/lib/common.sh"
INSTALL_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"

step "Восстановление прав сервисов"
require_sudo

declare -a paths=(
    "services/grafana/data:472:472:Grafana"
    "services/influxdb/data:1000:1000:InfluxDB"
    "services/mqtt-broker/data:1883:1883:Mosquitto (data)"
    "services/mqtt-broker/log:1883:1883:Mosquitto (log)"
    "services/portainer/data:root:root:Portainer"
    "services/uptime-kuma/data:root:root:Uptime Kuma"
    "services/node-red/data:1000:1000:Node-RED"
    "services/apprise/config:1000:1000:Apprise (config)"
)

for entry in "${paths[@]}"; do
    IFS=':' read -r path uid gid name <<< "$entry"
    if [[ -d "$INSTALL_DIR/$path" ]]; then
        sudo chown -R "${uid}:${gid}" "$INSTALL_DIR/$path"
        success "$name → ${uid}:${gid}"
    fi
done

success "Права восстановлены. Запустите: cd $INSTALL_DIR && docker compose up
-d"
```

Дайте права: chmod +x scripts/restore.sh.

Использование:

```
bash
tar -xzf backup.tar.gz -C ~
cd ~/elis_vulcan-90
./scripts/restore.sh
docker compose up -d
```

7.3. Восстановление после потери сервера

Сценарий: сервер вышел из строя, есть только бэкап и архив образов.

На новом сервере:

```
bash
# 1. Установить Docker (см. раздел 3.1)
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker

# 2. Загрузить образы из архива
docker load < ~/full-stack-*.tar.gz

# 3. Распаковать полный бэкап
sudo tar -xzf ~/elis-vulcan-FULL-BACKUP-*.tar.gz -C ~
sudo chown -R $USER:$USER ~/elis_vulcan-90/

# 4. Восстановить права сервисов
cd ~/elis_vulcan-90
./scripts/restore.sh

# 5. Запустить стек
docker compose up -d
sleep 25
docker compose ps
```

7.4. Перенос на другой сервер через флешку

На исходном сервере:

```
bash
# 1. Примонтировать флешку
lsblk                                    # найти sdb1
sudo mkdir -p /mnt/usb
sudo mount /dev/sdb1 /mnt/usb

# 2. Скопировать проект
sudo rsync -av --progress ~/elis_vulcan-90/ /mnt/usb/elis_vulcan-90/

# 3. Скопировать образы
sudo cp ~/elis-vulcan-backups/full-stack-*.tar.gz /mnt/usb/

# 4. Синхронизировать и отмонтировать
sync
sudo umount /mnt/usb
```

На новом сервере:

```
bash
# 1. Примонтировать флешку
sudo mount /dev/sdb1 /mnt/usb

# 2. Скопировать проект
cp -r /mnt/usb/elis_vulcan-90 ~/

# 3. Загрузить образы
docker load < /mnt/usb/full-stack-*.tar.gz

# 4. Восстановить права
cd ~/elis_vulcan-90
./scripts/restore.sh

# 5. Запустить
docker compose up -d
```

7.5. Перенос через образы контейнеров

Сценарий: нужно перенести **только образы** (без данных) — например, на другой сервер для теста.

На исходном сервере:

```
bash
cd ~/elis_vulcan-90

# 1. Сохранить образы
docker save $(docker compose config --images | sort -u) \
    | gzip > /tmp/elis-images-$(date +%F).tar.gz

# 2. Перенести на новый сервер
scp /tmp/elis-images-*.tar.gz user@new-server:/tmp/
```

На новом сервере:

```
bash
# 1. Загрузить образы
docker load < /tmp/elis-images-*.tar.gz

# 2. Проверить
docker images | grep -E "frangoteam|eclipse|influx|grafana|telegraf|node-red|
apprise|uptime|portainer|drawio|nginx|logic-engine"
# Ожидается: 12 образов

# 3. Развернуть проект (только скрипты и compose)
git clone https://github.com/your-org/elis_vulcan_90.git
cd elis_vulcan_90
./install.sh --skip-docker
```

РАЗДЕЛ 8. УДАЛЕНИЕ СИСТЕМЫ

8.1. Три режима удаления

Режим	Команда	Что удаляет	Что сохраняет
Мягкий	./uninstall.sh --keep-data	Только контейнеры и сеть	Всё: данные, .env, скрипты
Полный	sudo ./uninstall.sh	Всё, включая директорию	Docker-образы и архивы вне проекта
Агрессивный	sudo ./uninstall.sh --purge	Всё без подтверждений	Docker-образы

8.2. Мягкое удаление (`--keep-data`)

Используется для: **обновления проекта без потери данных.**

```
bash
cd ~/elis_vulcan-90
./uninstall.sh --keep-data
```

Что произойдёт:

- Остановит и удалит 12 контейнеров.
- Удалит сеть `elis_vulcan-90_default`.
- **Сохранит** директорию проекта, данные FUXA, `.env`, `.secrets`

Восстановление после мягкого удаления:

```
bash
cd ~/elis_vulcan-90
docker compose up -d
sleep 25
docker compose ps
```

8.3. Полное удаление

⚠️ *Обязательно с* ***`sudo`*** *— иначе упадёт на root-файлах (****`mosquitto.db`****,* ***`grafana.db`****,* ***`portainer.db`****).*

```
bash
cd ~/elis_vulcan-90
sudo ./uninstall.sh
# Спросит дважды "Продолжить?" — оба раза y
```

Что произойдёт:

- Остановит и удалит контейнеры.
- Удалит все данные (services/*/data/).
- Удалит .env, .secrets-*.txt, .htpasswd-password.
- **Удалит директорию** ~/elis_vulcan-90/ целиком.

Что сохранится:

- Docker-образы (12 штук).
- Архивы в ~/elis-vulcan-backups/ (вне проекта).
- Архивы в ~/ (вне проекта).

8.4. Проверка после удаления

```
bash
echo "═══ 1. Директории нет ═══"
ls -la ~/elis_vulcan-90/ 2>&1

echo "═══ 2. Контейнеров нет ═══"
docker ps -a

echo "═══ 3. Сети нет ═══"
docker network ls | grep elis || echo "Сети нет"

echo "═══ 4. Образы остались ═══"
docker images | grep -cE "frangoteam|eclipse|influx|grafana|telegraf|node-red|apprise|uptime|portainer|drawio|nginx|logic-engine"
# Ожидается: 12
```

8.5. Если `uninstall.sh` упал на Permission denied

Быстрое решение:

```
bash
cd ~
sudo rm -rf ~/elis_vulcan-90
```

Правильное решение — патч `uninstall.sh` (автоматическое использование `sudo`):

В функции `remove_data()` замените:

```
bash
[[ -d "$INSTALL_DIR/$dir" ]] && rm -rf "$INSTALL_DIR/$dir"
```

На:

```
bash
if [[ -d "$INSTALL_DIR/$dir" ]]; then
    log "  Удаление: $dir"
    if ! rm -rf "$INSTALL_DIR/$dir" 2>/dev/null; then
        sudo rm -rf "$INSTALL_DIR/$dir" || warn "  Не удалось удалить $dir"
    fi
fi
```

РАЗДЕЛ 9. МОНИТОРИНГ И ДИАГНОСТИКА

9.1. Uptime Kuma

Откройте `http://<IP>:3001`, создайте администратора и добавьте HTTP-мониторинги для всех ключевых точек:

Сервис	URL
FUXA	`http://hmi-frontend:1881`
InfluxDB	`http://influxdb:8086/health`
Grafana	`http://grafana:3000/api/health`
Nginx	`http://nginx-auth:80` (ожидаем 401)
Apprise	`http://apprise:8000`

Пороговые значения: **интервал 60 с, таймаут 10 с, 3 повторные попытки.**

9.2. Apprise — центр уведомлений

Откройте `http://<IP>:8000`, добавьте получателей (Telegram Bot, Email, Slack). Из Uptime Kuma укажите URL Apprise в качестве webhook для оповещений.

9.3. Диагностические команды

Что проверяем	Команда
Здоровье стека	`docker compose ps`
Логи всех сервисов	`docker compose logs --tail=50`
Ресурсы контейнеров	`docker stats --no-stream`
USB внутри FUXA	`docker exec hmi_frontend ls -l /dev/tty*`
MQTT-брокер жив	`docker exec mqtt_broker mosquitto_sub -t '$SYS/broker/version' -C 1`
Запись в InfluxDB	`docker compose logs telegraf | grep -i error`

9.4. Проверка HTTP всех сервисов

```bash
for check in "FUXA:1881" "Grafana:3000" "InfluxDB:8086" "Draw.io:8080" "Node-RED:1880" "Apprise:8000" "Uptime:3001"; do
    name="${check%%:*}"; port="${check##*:}"
    code=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "http://localhost:${port}")
    printf "  %-12s → HTTP %s\n" "$name" "$code"
done
```

РАЗДЕЛ 10. УПРАВЛЕНИЕ ПОЛЬЗОВАТЕЛЯМИ И БЕЗОПАСНОСТЬ

10.1. Пользователи FUXA

Включение аутентификации:

1. Войти в FUXA (http://<IP>:1881).
2. **Setup (шестерёнка) → Settings → System**.
3. Включить Authentication with Token (secureEnabled: true).
4. **НЕ включать** userRole — это ломает маршрутизацию (Cannot GET /userRoles).
5. Сохранить, перезапустить docker compose restart hmi-frontend.
6. Логин по умолчанию: admin / 123456.
7. **СРАЗУ сменить пароль** в **Setup → Users**.

⚠️ ***Критическая уязвимость:*** *FUXA версий* ***до 1.3.2*** *содержат обход аутентификации (CVE-2025-69985, CVE-2026-13207).* ***Срочно обновите*** *образ до версии 1.3.3+:*

```
bash
cd ~/elis_vulcan-90
nano docker-compose.yml
# Заменить frangoteam/fuxa:latest → frangoteam/fuxa:1.3.3
docker compose pull hmi-frontend
docker compose up -d hmi-frontend
```

Если `Cannot GET /userRoles`:

```
bash
cd ~/elis_vulcan-90
docker compose stop hmi-frontend
sudo mv services/hmi-frontend/appdata/mysettings.json \
        services/hmi-frontend/appdata/mysettings.json.bak
docker compose start hmi-frontend
```

10.2. Пользователи Portainer

При первом входе `https://<IP>:9443` создайте администратора. ⚠️ *Окно создания — 5 минут**, потом Portainer блокирует создание.***

Если не успели:

```
bash
cd ~/elis_vulcan-90
docker compose stop portainer
docker compose rm -f portainer
sudo rm -rf services/portainer/data/*
docker compose up -d portainer
# Сразу открыть https://<IP>:9443
```

10.3. Базовая авторизация Draw.io

Файл `services/nginx/.htpasswd` управляет доступом. Пароль — в `.htpasswd-password`.

Добавить пользователя:

```
bash
docker run --rm -it httpd:alpine htpasswd -nB <username> >>
services/nginx/.htpasswd
```

Сменить пароль:

```
bash
htpasswd -bB services/nginx/.htpasswd admin '<новый_пароль>'
docker compose restart nginx-auth
```

10.4. Общие рекомендации

- **Не открывайте наружу** порты 8086, 3000, 1880, 8000, 3001, 9000, 9443 — они для внутренней сети или VPN.
- Для удалённого доступа — VPN (WireGuard / OpenVPN) или reverse-proxy с TLS.
- Регулярно меняйте пароли администраторов InfluxDB, Grafana, Portainer.
- **API-токен InfluxDB** — ключ ко всем данным. Храните только в .env.

10.5. Чек-лист безопасности

- **FUXA:** версия ≥ 1.3.3 (иначе уязвимость обхода аутентификации).
- **Пароли:** ≥ 12 символов, буквы+цифры+спецсимволы.
- `.env`**:** не в git, не в открытых каналах.
- **Порты:** не открывать наружу (кроме 1881, 8080, 9443 — за VPN).
- **API-токен InfluxDB:** только в `.env`.
- **SSH:** использовать ключи вместо паролей.
- **Firewall:** `ufw` настроен.
- **Обновления:** образы обновляются регулярно.

10.6. Чек-лист при передаче сервера

Операция / Действие	**Команда для выполнения**	**Описание и примечание**
1. Установка	./install.sh	Запуск мастера автоматической установки системы.
2. Проверка статуса	docker compose ps	Проверка здоровья стека. Должно быть **12/12 Up**.
3. Полный бэкап данных	sudo tar -czf ~/backup-$(date +%F).tar.gz -C ~ elis_vulcan-90/	Создание архива директории проекта с правами `sudo`.
4. Бэкап Docker-образов	docker save $(docker compose config --images) \| gzip > ~/full-stack-$(date +%F).tar.gz	Создание «золотого слепка» всех 12 образов для офлайн-переноса.
5. Мягкое удаление	./uninstall.sh --keep-data	Удаление только контейнеров и сети с сохранением всех данных.
6. Восстановление / Запуск	docker compose up -d	Быстрый запуск системы из существующих конфигураций.
7. Полное удаление	sudo ./uninstall.sh	Безвозвратное удаление системы и всех накопленных данных.

РАЗДЕЛ 11. ОБНОВЛЕНИЕ СИСТЕМЫ

11.1. Обновление образов

```
bash
cd ~/elis_vulcan-90

# 1. Бэкап ПЕРЕД обновлением
sudo tar -czf ~/backup-before-update-$(date +%F).tar.gz -C ~ elis_vulcan-90/

# 2. Обновить
docker compose pull
docker compose up -d

# 3. Проверить
docker compose ps
```

11.2. Обновление Python-модуля

```
bash
docker compose up -d --build logic-engine
```

Скрипты монтируются из `services/logic-engine/scripts/`, поэтому при их правке достаточно `docker compose restart logic-engine`.

11.3. Обновление «золотого слепка» образов

```
bash
cd ~/elis_vulcan-90
docker save $(docker compose config --images) | gzip > ~/elis-vulcan-
backups/full-stack-$(date +%F).tar.gz
ls -lh ~/elis-vulcan-backups/
```

РАЗДЕЛ 12. ТИПОВЫЕ НЕИСПРАВНОСТИ

12.1. FUXA не видит PLC

Симптом: в FUXA Devices → Modbus → Test Connection красный.

Решение:

```
bash
ls -l /dev/arduino
docker exec hmi_frontend ls -l /dev/tty*
sudo udevadm trigger
docker compose restart hmi-frontend
```

12.2. FUXA не подключается к MQTT

Симптом: статус MQTT-устройства в FUXA красный.

Решение: в настройках MQTT-устройства проверьте, что **Server** = `mqtt-broker` (не `mosquitto`, не `localhost`).

12.3. FUXA `Cannot GET /userRoles`

Симптом: ошибка `Cannot GET /userRoles` после включения `userRole`.

Решение:

```
bash
cd ~/elis_vulcan-90
docker compose stop hmi-frontend
sudo mv services/hmi-frontend/appdata/mysettings.json \
        services/hmi-frontend/appdata/mysettings.json.bak
docker compose start hmi-frontend
```

Причина: `userRole: true` ломает маршрутизацию в бесплатной версии FUXA.

12.4. Telegraf не пишет в InfluxDB

Симптом: в логах Telegraf `unauthorized` или `token invalid`.

Решение:

1. InfluxDB (:8086) → **Data** → **API Tokens** → **Generate All-Access**.
2. Скопировать токен в services/telegraf/.env → INFLUXDB_TOKEN=.
3. docker compose restart telegraf.

12.5. Grafana Permission denied

Симптом: после восстановления из tar или перезапуска Grafana падает с `permission denied`.

Решение:

```
bash
sudo chown -R 472:472 services/grafana/data/
docker compose restart grafana
```

*Профилактика: **всегда используйте `sudo` при создании/распаковке бэкапа и запускайте `scripts/restore.sh` после `tar`.***

12.6. Portainer не даёт создать админа

Симптом: окно создания не открывается, прошло >5 минут.

Решение: перезапустить или сбросить (см. раздел 10.2).

12.7. Контейнер не запускается из-за `devices`

Симптом: ошибка `error gathering device information while adding custom device /dev/arduino`.

Решение: PLC не подключён или udev-правило не применено. Подключите устройство, дождитесь появления `/dev/arduino`, либо временно закомментируйте блок `devices:` в сервисе `hmi-frontend`.

12.8. Стек падает из-за занятого порта

Симптом: `bind: address already in use`.

Решение:

```
bash
sudo ss -tulpn | grep <port>
# Остановить процесс или изменить порт в docker-compose.yml
```

12.9. `install.sh` падает на `generate_password` (Exit 141

Симптом: `Exit code: 141` в `04-generate-secrets.sh`.

Причина: `set -o pipefail` + SIGPIPE от `head`.

Решение: в `scripts/lib/common.sh` заменить `generate_password` на:

```
bash
generate_password() {
    local len="${1:-24}"
    head -c 1024 /dev/urandom | tr -dc 'A-Za-z0-9!@#%^&*' | head -c "$len"
}
```

12.10. Проблемы с правами после `tar`

Симптом: после `tar -xzf` Grafana, Mosquitto, Portainer падают с `Permission denied`.

Причина: `tar` не сохраняет UID/GID файлов.

Решение: запустить `./scripts/restore.sh` после распаковки.

12.11. Не работает USB-монтирование

Симптом: `lsblk` не видит флешку, или `mount` падает.

Решение:

```
bash
# 1. Проверить, что ядро увидело устройство
dmesg | tail -20

# 2. Найти устройство
lsblk -f

# 3. Создать точку монтирования
sudo mkdir -p /mnt/usb

# 4. Примонтировать (с указанием ФС)
sudo mount -t vfat /dev/sdb1 /mnt/usb   # FAT32
sudo mount -t exfat /dev/sdb1 /mnt/usb  # exFAT
```

```
sudo mount -t ntfs-3g /dev/sdb1 /mnt/usb # NTFS

# 5. Отмонтировать перед извлечением
sudo umount /mnt/usb
```

Если `exfat` не поддерживается:

```
bash
sudo apt install -y exfat-fuse exfatprogs
```

РАЗДЕЛ 13. ПРИЛОЖЕНИЯ

13.1. Быстрые команды

```
bash
# Показать все контейнеры
docker compose ps

# Логи конкретного сервиса
docker compose logs -f hmi-frontend --tail=100

# Перезапустить всё
docker compose restart

# Полный рестарт
docker compose down && docker compose up -d

# Проверить HTTP всех сервисов
for check in "FUXA:1881" "Grafana:3000" "InfluxDB:8086" "Draw.io:8080" "Node-
RED:1880" "Apprise:8000" "Uptime:3001"; do
    name="${check%%:*}"; port="${check##*:}"
```

```
    code=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5
"http://localhost:${port}")
    printf "  %-12s → HTTP %s\n" "$name" "$code"
done
```

13.2. URL всех сервисов

Сервис	URL	Логин / Пароль
FUXA	`http://<IP>:1881`	`admin` / (свой после установки)
Draw.io	`http://<IP>:8080`	`admin` / (из `.htpasswd-password`)
InfluxDB	`http://<IP>:8086`	`admin` / (из `.secrets-*.txt`)
Grafana	`http://<IP>:3000`	`admin` / (из `.secrets-*.txt`)
Node-RED	`http://<IP>:1880`	без авторизации
Apprise	`http://<IP>:8000`	без авторизации
Uptime Kuma	`http://<IP>:3001`	создать при входе
Portainer	`https://<IP>:9443`	создать при входе

13.3. Чек-лист полного цикла

Развертывание и проверка системы

1. Автоматическая установка:
   `./install.sh`
2. Диагностика состояния (ожидаемый статус всех 12 контейнеров — **Up**):
   `docker compose ps`

- **Резервное копирование (Бэкап)**

  3. Создание полного бэкапа конфигурации и баз данных (требуются права `sudo`):
  `sudo tar -czf ~/backup-$(date +%F).tar.gz -C ~ elis_vulcan-90/`
  4. Создание «золотого слепка» всех 12 Docker-образов для работы без интернета:
  `docker save $(docker compose config --images) | gzip > ~/full-stack-$(date +%F).tar.gz`
- **Восстановление и миграция**

  5. Быстрый запуск / восстановление работы контейнеров:
  `docker compose up -d`
  6. Восстановление всей системы из tar-архива с автоматическим накатом прав:
  `tar -xzf backup.tar.gz -C ~ && ./scripts/restore.sh && docker compose up -d`

7. Развертывание системы на новом сервере из автономного архива образов:
`docker load < full-stack.tar.gz && docker compose up -d`

- **Демонтаж и удаление**

  8. Мягкое удаление (удаляет только контейнеры и сеть, **сохраняя** данные и `.env`):
  `./uninstall.sh --keep-data`
  9. Полное удаление (безвозвратная очистка сервера от всех компонентов и баз данных):
  `sudo ./uninstall.sh`

13.4. Глоссарий

Термин	Определение
SCADA	Supervisory Control And Data Acquisition — диспетчерское управление и сбор данных
HMI	Human-Machine Interface — человеко-машинный интерфейс
IIoT	Industrial Internet of Things — промышленный интернет вещей
MQTT	Message Queuing Telemetry Transport — протокол обмена сообщениями
Modbus RTU	Промышленный протокол для последовательных линий
TSDB	Time Series Database — база данных временных рядов
OPC UA	Open Platform Communications Unified Architecture — промышленный протокол
udev	Менеджер устройств Linux
CVE	Common Vulnerabilities and Exposures — идентификатор уязвимости
HA	High Availability — высокая доступность

13.5. Контакты и поддержка

- **Проект:** ELIS_Vulcan-90
- **Репозиторий:** https://github.com/SistemaTehnik/elis_vulcan_90
- **Документация:** `~/elis_vulcan-90/docs/`
- **Telegram:** @Zhu_ch_kov

Документ распространяется под лицензией MIT. См. файл LICENSE в репозитории.